



```
        b[i] == b[j] == b[k] == letter
    for i, j, k in winning_conditions
)
```

```
def is_board_full(b):
    return ' ' not in b
```

```
def get_empty_cells(b):
    return [i for i, spot in enumerate(b) if spot == ' ']
```

```
# Minimax
```

```
def minimax(b, is_maximizing):
```

```
    if is_winner(b, 'O'):
        return 1
```

```
    if is_winner(b, 'X'):
        return -1
```

```
    if is_board_full(b):
        return 0
```

```
    if is_maximizing:
        best_score = -math.inf
```

```
        for cell in get_empty_cells(b):
```



```
-L-----J-  
-  
score = minimax(b, False)  
b[cell] = ' '  
best_score = max(best_score, score)  
  
return best_score  
  
else:  
    best_score = math.inf  
  
    for cell in get_empty_cells(b):  
        b[cell] = 'X'  
        score = minimax(b, True)  
        b[cell] = ' '  
        best_score = min(best_score, score)  
  
    return best_score  
  
def ai_move(b):  
    global path_cost  
  
    best_score = -math.inf  
    move = None  
  
    for cell in get_empty_cells(b):  
        b[cell] = 'O'  
        score = minimax(b, False)  
        b[cell] = ' '
```

5]
5s



```
        if score > best_score:
            best_score = score
            move = cell

    b[move] = 'O'
    path_cost += 1

    print("AI selected position:", move)
    print("Path Cost:", path_cost)

def player_move(b):
    global path_cost

    while True:
        try:
            choice = int(input("Enter position (0-8): "))

            if choice < 0 or choice > 8:
                print("Enter a number between 0 and 8.")
                continue

            if b[choice] == ' ':
                b[choice] = 'X'
                path_cost += 1
                print("Path Cost:", path_cost)
                break

        else:
```



```
        print("That position is already occupied!")
```

```
    except ValueError:  
        print("Invalid input.")
```

```
# Main  
print("TIC-TAC-TOE")  
print("You are X and AI is O")  
  
print("\nInitial State:")  
print_board(board)  
print("Initial Path Cost:", path_cost)
```

```
while True:
```

```
    # Player move  
    player_move(board)  
    print_board(board)  
  
    if is_winner(board, 'X'):  
        print("You win!")  
        break
```

```
    if is_board_full(board):  
        print("It's a tie!")  
        break
```

```
    # AI move
```

] 5s



```
        break

    # AI move
    print("AI's turn...")
    ai_move(board)
    print_board(board)

    if is_winner(board, 'O'):
        print("AI wins!")
        break

    if is_board_full(board):
        print("It's a tie!")
        break
```

... TIC-TAC-TOE
You are X and AI is O

Initial State:

O				X
---		---		---
X				
---		---		---
X		O		O

TIC-TAC-TOE

You are X and AI is O

...

Initial State:

O				X
---		---		---
X				
---		---		---
X		O		O

Initial Path Cost: 0

Enter position (0-8): 5

Path Cost: 1

O				X
---		---		---
X				X
---		---		---
X		O		O

AI's turn...

AI selected position: 4

Path Cost: 2

O				X
---		---		---
X		O		X
---		---		---
X		O		O

AI wins!



```
if is_board_full(board):  
    print("It's a tie!")  
    break
```



TIC-TAC-TOE

You are X and AI is O

Initial State:

O				X
---		---		---
X				
---		---		---
X		O		O

Initial Path Cost: 0

Enter position (0-8): 4

Path Cost: 1

O				X
---		---		---
X		X		
---		---		---
X		O		O

You win!

TIC-TAC-TOE

You are X and AI is O

Initial State:

O				X
---		---		---
X				
---		---		---
X		O		O

Initial Path Cost: 0

Enter position (0-8): 1

Path Cost: 1

O		X		X
---		---		---
X				
---		---		---
X		O		O

AI's turn...

AI selected position: 4

Path Cost: 2

O		X		X
---		---		---
X		O		
---		---		---
X		O		O

AI wins!